

NLP-Based Sentiment Analysis of Movie Reviews Using TF-IDF and Logistic Regression

Mini NLP Project Report

Course Outcomes Addressed:CO3,CO4

Submitted By:

Name: Abhishek V Nair
Reg No: 24UBC103
III BCA A

Submitted To:

Name:Ms. Rosamma K S
BCA Department

Git Hub REPO

Link:<https://github.com/Abhishek-zy/NLP-Sentiment-Analysis-Project>

1. Problem Statement

The online forums create a vast amount of content from users in the form of reviews. It takes a long time for manually going through such texts and categorizing them in order to gauge the public opinion. This project aims at addressing the issue of automatically predicting the sentiment (negative/positive) of a movie review using NLP and Machine Learning algorithms.

It is an NLP problem since it involves transformation of the unstructured human-readable text into structured numeric data which the machine learning model can understand and use to predict the sentiment of the text.

2. Objectives

- To collect and preprocess a large dataset of real movie reviews.
- To convert raw text into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency).
- To train a Logistic Regression classifier to predict review sentiment (positive/negative).
- To evaluate the trained model using standard classification metrics.
- To analyze which words most strongly influence the model's predictions.
- To identify the limitations of the approach and suggest future improvements.

3. Dataset

Dataset Name: IMDB Dataset of 50K Movie Reviews

Attribute	Details
Source	Kaggle (uploaded by Lakshmi Narayan Pathi), accessed via the kagglehub library
Size	50,000 movie reviews
Fields	review (raw text of the review), sentiment (label: positive / negative)
Target Variable	sentiment
Number of Classes	2 (Positive, Negative)
Class Balance	25,000 positive and 25,000 negative reviews (perfectly balanced)
Limitations	Only covers movie reviews (not other domains); labels are binary only, with no neutral class; does not capture sarcasm well

4. Methodology

Raw Reviews → *Text Preprocessing* → *TF-IDF Feature Extraction* → *Logistic Regression Model* → *Prediction* → *Evaluation*

4.1 Text Preprocessing

Raw reviews contain HTML tags, punctuation, numbers, and common words (stopwords) that add noise without contributing meaningfully to sentiment. Each review was processed as follows:

- Converted to lowercase for uniformity.
- HTML tags (e.g.
) and URLs removed.

- Punctuation and numbers removed.
- Common stopwords (e.g. "the", "is", "was") removed using NLTK's stopwords list.
- Extra whitespace collapsed.

4.2 TF-IDF Feature Extraction

The TF-IDF method transforms each preprocessed review into a numeric vector. It gives more weight to those words that are frequently found in that particular review but not in the whole dataset, and lesser weight to words that are often found in almost all reviews. In this way, the model pays attention to those words which truly indicate the sentiment of the review, such as "brilliant" and "boring". The maximum number of features selected by the vectorizer is 10,000 (max_features=10000).

4.3 Model: Logistic Regression

Logistic Regression is used as the classifier. Each word or feature in the dataset has a learned weight by the algorithm, which determines how much that particular word affects the result to either classify it as positive or negative. Logistic Regression was preferred over other algorithms such as deep learning since it trains faster and is more understandable because the weights learned by it can easily be inspected. Furthermore, it suits well to the sparse linearly separable high dimensional feature space of TF-IDF vector space.

5. Implementation

Component	Details
Programming Language	Python 3
Libraries Used	pandas, scikit-learn, nltk, matplotlib, seaborn, kagglehub
NLP Technique	Text cleaning + stopwords removal, TF-IDF vectorization
Algorithm	Logistic Regression (scikit-learn)
Train/Test Split	80% training (40,000 reviews) / 20% testing (10,000 reviews), stratified

The complete, commented source code is included in the submitted GitHub repository (source_code.py).

6. Results

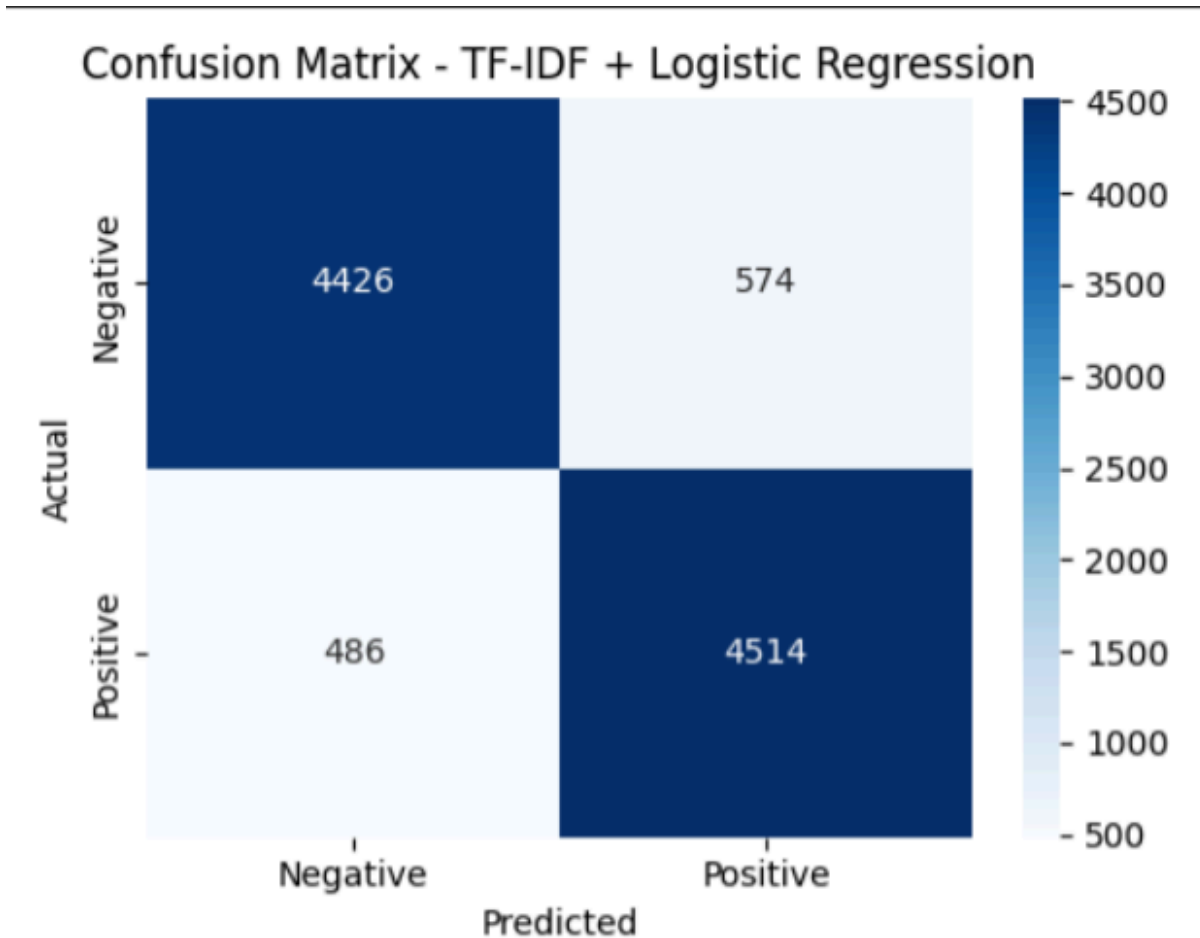
The trained model was evaluated on the held-out 10,000-review test set. The results obtained were:

Metric	Score
Accuracy	89.40%
Precision	88.72%
Recall	90.28%
F1-score	89.49%

6.1 Confusion Matrix

The table below shows the model's predictions broken down by actual class (10,000 test reviews: 5,000 actual positive, 5,000 actual negative).

	Predicted: Negative	Predicted: Positive
Actual: Negative	4,425 (True Negative)	575 (False Positive)
Actual: Positive	486 (False Negative)	4,514 (True Positive)



6.2 Most Influential Words

Inspecting the learned Logistic Regression weights shows which words most strongly push a prediction toward each class, confirming that the model learned linguistically sensible patterns rather than arbitrary noise.

Top Positive Words	Top Negative Words
great, excellent, perfect	worst, waste, awful
amazing, best, loved	bad, boring, terrible
wonderful, hilarious, brilliant	poor, dull, horrible

6.3 Sample Predictions

Review	Predicted Sentiment	Confidence
"This movie was an absolute masterpiece, brilliant acting and story."	Positive	0.95
"Waste of time. Boring plot and terrible acting."	Negative	1.00

7. Limitations

- The model only handles English-language movie reviews and may not generalize well to other domains (e.g. product reviews) without retraining.
- TF-IDF treats text as a "bag of words" and ignores word order, so it cannot capture context, negation nuances, or sarcasm well (e.g. "not good" vs "good").
- Sentiment is restricted to two classes (positive/negative); there is no handling of neutral or mixed sentiment.
- The vocabulary is capped at 10,000 words, so rare or highly specific terms outside this vocabulary are ignored.
- Performance may degrade on very short reviews or reviews with heavy slang/spelling errors not well represented in the training data.

8. Future Scope

- Extend the dataset to other domains (e.g. product or restaurant reviews) to test generalization.
- Introduce a neutral/mixed sentiment class for finer-grained predictions.
- Explore word-embedding or transformer-based approaches (e.g. Word2Vec, BERT) to better capture context and word order.
- Deploy the model as a simple web application where users can input a review and get real-time sentiment feedback.
- Improve preprocessing to better handle negation, emojis, and informal/slang text.

9. Conclusion

The proposed model is a sentiment classification model that classifies whether a review is a positive one or a negative one, utilizing TF-IDF for feature extraction and Logistic Regression for classification. The training and testing of the model were performed using the IMDB 50K Movie Reviews Dataset, where the model gave an accuracy score of 89.40% and F1-score of 89.49% on unseen testing data. The weights learned by the model proved to be intuitive, and the model correctly classified the sentiments of manually created test sentences. Thus, this project shows that even a simple and well-known natural language processing pipeline (text cleaning, TF-IDF vectorization, and linear classifier) can produce excellent results on a real-life text classification task.